

Revealing Regressions: A Comparative Study of State-Capture Strategies in Validating Program Behavior

HANG DU, University of California Irvine, USA

VIJAY KRISHNA PALEPU, Microsoft, USA

JAMES A. JONES, University of California Irvine, USA

A central challenge in software testing is deciding which parts of a program's state to check as evidence of correct behavior to reveal regressions. These checks are embodied as test oracles, typically as assertions in test cases. State observation strategies play a decisive role in shaping how effectively regressions can be revealed. Such strategies range from exhaustive memory snapshots to selective attribute checks via getter methods and nullability checks. These strategies are deeply embedded in both research and practice: academic work has explored heuristic- and serialization-based oracles, while industry has widely adopted snapshot testing. Despite their importance, the effects of different state-capture choices remain poorly understood from a scientific perspective. In this work, we present an experimental framework for systematically analyzing these design choices along the dimensions of observation scope, extraction approach, and extraction depth. Using this framework, we conduct an empirical study across twelve real-world projects, measuring how state-capture strategies influence regression-revealing capability and the richness of oracle information. Our findings reveal that human-written assertions often under-utilize available program state, achieving well below the fault-revealing potential of systematic observation strategies. Simple design choices, such as exposing unchecked intermediate return values, carefully selecting getters, and deepening state extraction, can yield measurable improvements (avg. 35.7%) in regression detection without needing to observe an overwhelmingly large amount of program-state data. Additionally, we highlight the challenges of observability, assertion desirability, and the trade-offs of capturing richer program states. Such insights show how small design choices can yield major differences in regression detection and potentially offer concrete directions for both tool builders and practitioners.

CCS Concepts: • **Software and its engineering** → **Software verification and validation**.

Additional Key Words and Phrases: program state observation, runtime monitoring, instrumentation, test oracle generation

ACM Reference Format:

Hang Du, Vijay Krishna Palepu, and James A. Jones. 2026. Revealing Regressions: A Comparative Study of State-Capture Strategies in Validating Program Behavior. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE079 (July 2026), 24 pages. <https://doi.org/10.1145/3797107>

1 Introduction

Capturing and reusing program states during execution is a long-standing practice in software testing to establish expected behavior for future validation. These observed behaviors act as *test oracles*, determining whether a program behaves as intended when executed.

Authors' Contact Information: [Hang Du](#), University of California Irvine, Irvine, USA, hdu5@uci.edu; [Vijay Krishna Palepu](#), Microsoft, Mountain View, USA, vijay.palepu@microsoft.com; [James A. Jones](#), University of California Irvine, Irvine, USA, jajones@uci.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE079

<https://doi.org/10.1145/3797107>

This practice has emerged from different communities and is described with varied terminology and in different contexts — such as “regression,” “differential,” “carving,” “snapshot,” or “observation-based.” Irrespective of how such testing practices are talked about, at their core, they all rest on the same principle: **using previously observed states as the benchmark for expected behavior**. Indeed, these approaches differ in how much state they capture, how they extract it, and how deeply they encode it into oracles. To provide a clearer picture of the landscape, we classify them into three families of state-capture-based oracle construction, highlighting how the idea has evolved across academic research and industrial practice.

- (1) **Heuristic-based regression oracles.** These approaches rely on directly accessible outputs, guided by intuitive or heuristic choices. They select parts of the program output — such as return values of method calls in a test method — and apply simple checks like nullability or `toString`. The resulting values are embedded in assertions as oracle data. Tools such as RANDOOP [53], EvoSUITE [27, 28], and DSpot [8, 17] exemplify this category, which remains active with ongoing research into assertion readability [9, 18], observability [30], and effectiveness.
- (2) **Serialization-based oracles.** These approaches capture more exhaustive program states to reuse them as a whole to reveal behavioral divergences. They are central to *differential testing* with *test carving* [23, 25, 46], where executions are decomposed into unit-level tests for replay. Unlike heuristic-based oracles, they typically rely on serializing and deserializing complete program states to construct carved tests and oracles. This serialization-based style is supported by various libraries [1, 39, 71] and has been used in research [22, 26, 47, 61, 66, 67, 69, 71] to construct oracles, enabling the systematic capture and reuse of complex states.
- (3) **Snapshot oracles in industry.** Conceptually similar to serialization-based oracles, this family is widely known in industry as *snapshot testing* [32]. It captures the *entire state* of program output—a “snapshot”, by its name—and reuses it wholesale as the oracle. Originally developed for UI testing, snapshot testing has since been extended to general program objects and adopted at scale by companies such as Airbnb, Lyft, and Robinhood [32]. For example, Meta’s TESTGEN [4] generates unit tests by carving from serialized observations of complex objects during app execution, which they describe as “observation-based unit test generation.” Despite its popularity in practice, snapshot testing has received little attention in academia, with one of the first explicit literature [32] appearing as recently as 2023.

In practice, these families often overlap, with hybrid techniques blending elements from each. Their idiosyncratic design choices directly influence oracle data—shaping its structure, fault-detection capability, brittleness, accessibility, and the amount of information used for verification. Yet, despite decades of research and extensive industrial use, the community still lacks a systematic understanding of how such seemingly small differences in state-observation strategies impact regression assertions’ fault-detection capabilities. To the best of our knowledge, no prior work has systematically compared the fault-revealing capabilities across the full spectrum of these strategies within a single, controlled study.

In this work, we propose a conceptual framework that characterizes state-capture strategies to reveal regressions along three dimensions—*state observing scope*, *state extraction approach*, and *state extraction depth*. Using this framework, we conduct an empirical study across twelve real-world open-source Java projects using mutation analysis to examine how these choices influence fault detection and the richness of oracle information they encode. Our results reveal how the sensitivity of state-capture-based oracles is shaped by design choices in representation and selection of program state. More broadly, the findings offer insights into how faults propagate to observable states and become detectable, advancing both automated oracle methods and practical testing practices in research and industry.

The main contributions of this paper are:

- A conceptual framework of design choices for capturing program memory state to detect regressions during test execution.
- An experimental design that systematically evaluates different program-state capture strategies for their sensitivity to fault-detection. Additionally, we offer an open-source replication artifact [2] of our experimental design, implementation, and data.
- Empirical insights into:

State Observing Scope: Our findings show that the elements and structure of the test prefix can complicate reliable regression detection. Validating return values in test prefix is highly valuable yet they are often left embedded in chained calls and unasserted in human-written tests. Making them explicit and asserting them in sequence can strengthen regression revealing capabilities, by on average over 30%. However, the conditional logic in test prefixes (*i.e.*, Conditional Logic Test Smell [3]) can silently alter execution logic, leaving up to 35% of faults undetected without proper assertions.

State Extraction Approach: While one might expect memory walks (*i.e.*, snapshots) to be the most effective approach, due to their exhaustive nature, getter-based extraction achieves similar capability with $10\times$ – $1000\times$ less oracle data. Yet including `toString` and `hashCode` as available getters is surprisingly impactful, with exclusions reducing detection by up to 30.5%, which highlights the challenges of state observability and assertion desirability.

State Extraction Depth: Simply using an additional getter step to access deeper program behaviors as oracle data can yield a further fault-revealing improvement of up to 7.7% compared to using one simple getter that returns primitive values.

2 Background

2.1 State-Capture within Test Cases

Unit tests consist of two major components: a *test prefix* and *test assertions*. The first component, *test prefix* [19], or *test data* [59] — denotes an initial sequence of inputs that transitions the system into a specific state. The second component, *test assertions*, use the memory state that was created by the test prefix to verify the values of specific memory locations. For example, Figure 1 depicts a JUnit test method from Apache Dubbo Common, which contains a test prefix in Lines 1–7 and a test assertion at Line 8.

Throughout this work, we use the term *program state* to refer specifically to the in-memory state of a Java program that is directly accessible from the test code’s variables via the JVM operand stack and memory heap at runtime. This includes object fields, return values, and transitively reachable references; however, note that it does not encompass external resources such as database tables, files on the filesystem, network connections, or GUI/console output, unless such data has been loaded into memory and is accessible through variables within the test code.

Checking Variables from Different Parts of the Test Prefix. The test prefix of most test cases provides a myriad opportunities and choices of which state to capture and observe in order to check for regressions. The test assertion at Line 8 in Figure 1 chooses to observe the value of the local variable merged to compares its value against an expected list of strings. However, this local variable is only one of many points of state observation that could alternatively or additionally be made. For example, the return values of `ApplicationModel.defaultModel()` or `Application.defaultModel().getExtensionDirector()` are both viable entry points to observe memory state. Moreover, the values that returned in each of these two hypothetical cases are likely object references that contain a memory graph of fields, some of which are object references themselves, each containing their own fields that have values that can be observed and compared to expected state.

```

1  @Test
2  void testMergeValues() {
3      List<String> merged = ConfigUtils.mergeValues(
4          ApplicationModel.defaultModel().
5              getExtensionDirector(),
6              ThreadPool.class,
7              "aaa,bbb,default.custom",
8              asList("fixed", "default.limited", "cached"));
9      assertEquals(asList("fixed", "cached", "aaa", "bbb", "
10                     default.custom"), merged);
11  }

```

Fig. 1. Example test method from which to observe state to potentially reveal regressions.

To give a sense for the variety of choices about which parts of the test prefix (*i.e.*, variables, or “root memory locations”) are used as the basis state observation, consider the following state-capturing practices in state-of-the-art research tools. EVOsuite [27–29] observes and captures return values of method calls. In Figure 1, the return state from each of the method calls in the test prefix would be captured to verify its state. Somewhat differently, ORSTRA [73] chooses to observe receiver objects, arguments, and return values for each method invocation in the prefix. In Figure 1, in addition to the return state from method calls, ORSTRA would observe the state of the parameter values and state passed into those method calls. In contrast, DSPOT [8, 17] focuses on local variables and those involved in method calls, DODONA [49] selects bytecode-level variables, asserting operands in assignments, and several other testing tools make their own set of choices of where to observe state.

Checking Variables with Different Specificity. Beyond simply prescribing which parts of the test prefix should be captured in order to observe its state, differing choices are made as to *how* and *how much* of that state is encoded and verified. Starting from the chosen root memory location described above, in order to extract and observe the state from that point, varying approaches have been taken. For example, DSPOT [8, 17] invokes getter methods on objects references that return primitive or string values. In another example, AGITARONE [57, 63] observes state using reflection to access private primitive fields on object references.

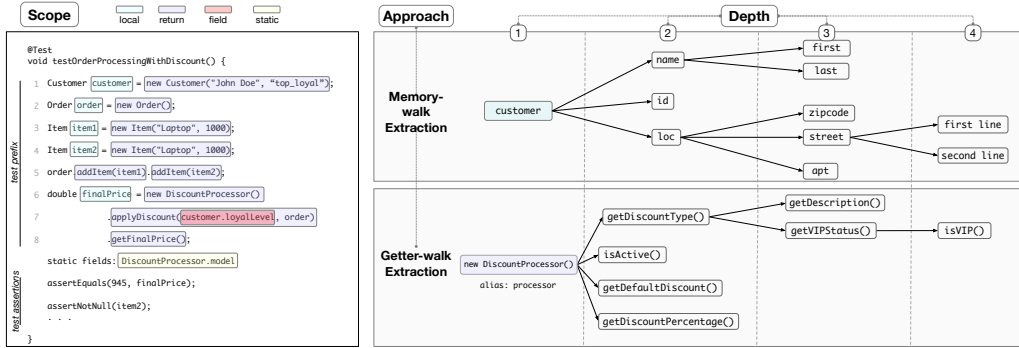
Moreover, state can be further observed on a range of scales from narrowly observing only primitive fields to more expansively walking out over all transitive object references to capture the entire memory graph, and at any intermediate threshold degree of expansion. One existing tool, AUTOASSERT [75] uses full serialization of an object and all of its transitive state to capture and compare state. Another tool, GASSERT [64, 65] uses a different approach by transitively traversing reachable primitive fields and primitive return values from getter methods.

In summary, these myriad design choices made by existing techniques for observing runtime state to detect regressions within test code demonstrate the degree of freedom and challenges that such tool builders must face. These choices include which (1) parts of a test prefix to target for state observation; (2) with what specificity; and (3) with what means to extract that state from the root memory location — all of which may affect the ability to detect regressions. To the best of our knowledge, no existing study systematically explores the fault-revealing capabilities of the large variety of state-capture strategies.

3 A Framework of State-Capture Choices

To enable the study of the effects of various design decisions for test-assertion generation, we devised a conceptual framework that characterizes these choices along three dimensions: (a) state-observing scope, (b) state-extraction approach, and (c) state-extraction depth.

The first dimension — state-observing scope — defines which parts of the test prefix or execution data can be asserted as discussed in Section 2. The state-extraction approach and depth determine the specificity of



(a) Existing test code with six lines of test prefix that highlights opportunities to observe state from four state scope types (left), and two state-extraction approaches at varying depths (right).

Assertion Strategy	Example Related Works	State-Observing Scope	State-Extraction Approach	Depth
Ⓐ assertEquals(deserialize(new File("oracleData.xml")), customer)	[4, 68, 69]	local	memory	4
Ⓑ assertEquals("Customer: {name:[Sam, Smith], id:12345, Loc:[73253, St.: [1 Main Rd., BLDG 1], 101]}", Serialize(customer))	[73, 75]	local	memory	4
Ⓒ assertNotNull(processor)	[27, 75]	return	memory/getter	1
Ⓓ assertEquals(12345, customer.getID())	[8, 27, 49, 53]	local	getter	2
Ⓔ assertEquals("50% off", customer.getDiscountType().getDescription())	[49]	local	getter	3
Ⓕ assertEquals(12345, customer.id)	[60, 64, 65]	local	memory	2
Ⓖ assertEquals("12345", (String)customer.getClass().getDeclaredField("id").get()) // use reflection	[60, 64, 65]	local	memory	2
Ⓗ assertEquals(278966883, customer.hashCode())	[8, 17, 53]	local	getter	2
Ⓙ assertEquals("Sam Smith", customer.toString())	[8, 17, 53]	local	getter	2
Ⓚ assertEquals(3, customer.loyalLevel) // customer.loyalLevel is the root when calculating depth	[27]	field	memory/getter	1
Ⓛ assertEquals(2, DiscountProcessor.model.getID()) // DiscountProcessor.model is the root when calculating depth	[21]	static	getter	2

(b) Example assertion types that can be generated to verify expected state.

Fig. 2. Example test prefix (a, left), state-extraction approaches (a, right), and generated assertion types (b).

variable checks within this scope. In addition to detailing each dimension, we also discuss the resulting size of the search space that is used as the oracle data in test assertions.

State-Observing Scope. A test prefix has direct access to heap or stack memory locations that are within the test code’s execution scope. We refer to such heap or stack memory locations as *state-observing scope*, and test-generation strategies can produce assertions for any of them directly, or use them as starting points for capturing and asserting additional program state.

The example test on the left in Figure 2a outlines multiple such locations in test code: (a) local variables, (b) method return values, (c) field dereferences, (d) static fields. Figure 2a depicts each such location type with a distinctly colored bounded box (e.g., all local variables are highlighted with a blue bounded box). Memory locations such as local variables, method return values, and results of field dereferences are typically stored (and available) on the test code’s operand stack. For example, Lines 6–8 involve three consecutive method calls with intermediate return values. However, these intermediate return values are transient in the operand stack while not directly being asserted in the assertions. As such, additional assertions may check intermediate variables like `new DiscountProcessor()` from the return-value scope. This may require explicitly referencing `new DiscountProcessor()` (e.g., an alias we give in the bottom right box of Figure 2a) and produces an assertion like assertion Ⓒ. Similarly, Line 7 dereferences a field `customer.loyalLevel` that is stored in the test code’s operand stack. In addition to operand stack value, publicly declared global variables (e.g., static fields in Java) are available as heap locations within the test execution (e.g., `DiscountProcessor.model` in Figure 2a).

State-Extraction Approach. State-observing scopes — stack and heap memory locations in the test methods’ execution scope — represent merely the start points, or the roots of the total program state that tests can capture and assert. The right side of Figure 2a presents the available memory states that are accessible from a given root: `customer` from the local variable scope and `new DiscountProcessor()` from the return-value scope.

Memory-walk Extraction. In the top right box of Figure 2a, the memory-state graph includes available memory locations by traversing object references and state. Such a “memory walk” extraction uses direct dereferences

of public object fields and/or a more complete traversal of the state using advanced state-capture techniques, such as reflection or object serialization (which often utilizes reflection). Assertions can verify specific fields, like in assertion ⑥ from Figure 2b; or use object serialization to capture all reachable fields from the root object, as shown in assertions ① and ②.

Getter-walk Extraction. In the bottom right box of Figure 2a, the state graph includes available, and potentially computed, memory locations by traversing getter (*i.e.*, inspector or accessor) methods. Such a “getter walk” extraction uses these inspector methods to iteratively and transitively traverse program state, even as some such states may be *computed* (rather than pre-stored) states. Examples of such assertions are shown in Figure 2b, *e.g.*, assertions ③, ④, ⑤, ⑥ and ⑦ in Figure 2b.

Among those, the inclusion of `toString` and `hashCode` in automated techniques is inconsistent among existing techniques and rarely discussed in prior research. Indeed, `toString` and `hashCode` methods may be defined by developers to represent the general memory-state within an object, typically used to define an object’s identity and equality relations with other objects. As such we speculate that usage of `toString` and `hashCode` may impact assertions’ sensitivity to faults, more than other methods in getter-walks. Given the inconsistent use of these methods in existing tools, our study considers the getter-walk approach both with and without `toString` and `hashCode`.

Both of these state-extraction approaches start their walk of the program’s state at the root memory locations from each of the state-observing scopes, and from there, are able to access programmer-defined object attributes that are not explicitly referenced in the test code.

State-Extraction Depth. State-extraction depth specifies the number of steps in a program-state walk to access program state as part of a test execution, starting from a memory location in the state-observation scope. The right side of Figure 2a illustrates several such program-state walks of varying depths. For instance, `customer.name` and `customer.loc.street.firstline` are examples of memory-walks, whereas the other option, `discountProcessor.getDiscountType().getVIPStatus()`, is an example of a getter-walk. In the right portion of Figure 2a, each node in the memory graph represents an extracted memory value, either using a chain of getters or a memory-walk, from an initial state-observation scope (*e.g.*, test’s local variable). Specifically, the root of a program-state walk is labeled as depth 1, which is a heap or stack memory location from test’s initial observation scope. Examples given in assertions ④ in Figure 2b include two assertions that use getters to access object variable’s attributes at depth 3. Assertions ①, ②, ③ and ⑤ in Figure 2b use memory-walk state extraction approach with varying state-extraction depths, while assertions ③, ④, ⑤, ⑥, and ⑦ employ getter-walks. Generally, the deeper the program-state walks, the more potential fault-revealing capability the generated assertions are likely to have. However, getter-walk approaches may generally be considered to expose fewer additional implementation details as the extraction depth increases than memory-walk extraction approaches, because these accessor methods may be highly abstracted by developers [73].

Oracle Data Size for Assertions. Although prior work has described oracle data size as the number of output variables monitored—and, by extension, the number of resulting assertions in a test [31], we take a more comprehensive approach to reflect that not all assertions are of equal power. For example, both assertion ② and assertion ③ are written as single statements, yet ② checks only the ID attribute, whereas ③ validates all attributes of the customer object.

Thus, in this work, we define **oracle data size** as the number of finest-grained attributes referenced in assertions, corresponding to the number of nodes in the memory state graph (Figure 2a). For instance, assertion ① or ② in Figure 2b has an oracle data size of 11 because its expected value is a serialized or deserialized object that implicitly includes all 11 attributes reachable from the customer variable. This single assertion therefore encodes the same information as 11 separate assertions, each checking one attribute.

4 Experiment

Using the framework of state-capture strategies in test runs, which we describe in Section 3, we now detail our experiment design to examine the oracle sensitivity and oracle data size of monitoring varied program states from test code, when used in detecting faults. We first enumerate the research questions of our experiment along with our experiment’s subject programs, followed by our experiment’s design. We present the experimental results in Section 5.

4.1 Research Questions

RQ1: How do program states, captured using different state-capture strategies, vary in their fault-revealing capabilities?

- *RQ1-a*: Do the extracted program states differ in fault-revealing capability, when extracted using different state-extraction approaches: getter-walks and memory-walks?
- *RQ1-b*: Do the program states observed using different state-observing scopes, contribute differently to fault-revealing capability?
- *RQ1-c*: Do the program states observed at different state-extraction depths, contribute differently to fault-revealing capability?

We first study how different state-capture strategies observe program states from test code and how these observations influence fault-detection capability. Our analysis focuses on the three dimensions of state observation introduced in Section 3: scope, approach, and depth. In particular, we examine the dimension of scope (*RQ1-b*) across three layers of the human-written test prefix: (1) individual observed variables and values from specific parts of the test prefix, (2) object equalities across different parts of the test prefix, and (3) divergent control-flow paths induced by conditional logic in the test prefix.

Specifically, (2) corresponds to equality-based assertions across different variables in the test prefix, which appear in both human-written and machine-generated tests [27]. (3) reflects the presence of non-linear control flow in human-written test prefixes, where branching constructs are included in test-method bodies (e.g., *if-else*, loops, or *try-catch*) can alter execution logic. Such constructs are sometimes regarded as test smells [3], since unit tests are generally expected to remain simple and linear.

Together, these three layers capture representative ways in which test prefixes expose program states, allowing us to study their fault-revealing capabilities and their implications for designing test-assertion generation tools.

RQ2: How does the size of oracle data vary with state-extraction depths for each state-extraction approach? Next, we analyze the number of program states that we are able to capture with the differing state-extraction approach across all state-extraction depths. Each observed program state can be used as oracle data for a generated test assertion. As such, we refer to the number of observed program states as *oracle data size*. We record the size of the oracle data when using different state observation scopes (local variables, return values, fields, static fields), different state-extraction approaches (memory vs. getter) and varying depths (1–5).

In *RQ2*, we approach oracle data size as a proxy measure for the potential cost associated with a higher fault-detection capability. One way to detect more faults would be to capture a higher number of program states that collectively detect more faults — observing more program states gives greater opportunities for discovering propagated infection from the faults. Capturing higher numbers of program states can be achieved either by asserting a greater size of the memory graph through means such as serialization (i.e., serializing an object’s memory graph to a string) or generating several smaller-grained assertions. However, along with the benefit of detecting more faults, there may be costs associated with generating and maintaining an evermore increasing number of test assertions or larger oracle sizes. For state-capture strategies, we study oracle data size as a measure of cost and analyze it as a trade-off with the fault-detection abilities of captured state. Additionally, we also examine and report the actual runtime and storage costs of our implementation.

RQ3: To what extent can observing varying program states amplify human-written tests? We compare the fault-revealing capabilities of observed program states with the original oracles and assertions already in the test suites. If certain state-capture strategies identify program states that reveal more faults than existing assertions, it indicates amplification [16] opportunities for a given test suite.

Subject Programs. To answer these research questions, we studied test suites from 12 open-source Java projects. We refer to these as *human-written* test suites, as all selected projects predate the widespread adoption of AI-based test generation tools (post-2022). While we cannot guarantee that every line was manually authored, we found no evidence of computationally generated tests in these repositories at the time of conducting the work. The projects are selected based on the following criteria: (1) use of MAVEN as the build system; (2) use of JUNIT testing framework; (3) compatibility with JDK 8/11; and (4) assertions from any of the JUNIT, ASSERTJ, and HAMCREST libraries. Of these, 9 subjects are either from the DEFECTS4J real-bug database [41] or

Table 1. Subject Programs

Subject Project	version	LoC	#T	LC	#Mut	MS
commons-cli	1.7.0	1357	617	98.1%	821	92.4%
spotify-web-api	8.4.0	7026	322	79.5%	2056	85.9%
incubator-seata-core	2.0.0	3795	134	30.2%	367	55.6%
commons-validator	1.8.0	3051	578	78.9%	1707	89.4%
MyBatis	3.5.16	10478	1773	88.1%	4798	91.6%
commons-text	1.12.0	5794	1303	98.4%	4464	85.3%
commons-csv	1.11.0	1165	838	99.5%	724	96.1%
joda-money	1.0.3	1218	1473	92.9%	899	85.1%
dubbo-common	3.2.12	15095	957	64.9%	6241	79.9%
jsoup	1.18.1	9113	1148	84.7%	5022	75.0%
commons-codec	1.15	4451	1131	91.8%	3665	91.7%
commons-net	3.11.1	9198	500	39.1%	2238	76.8%

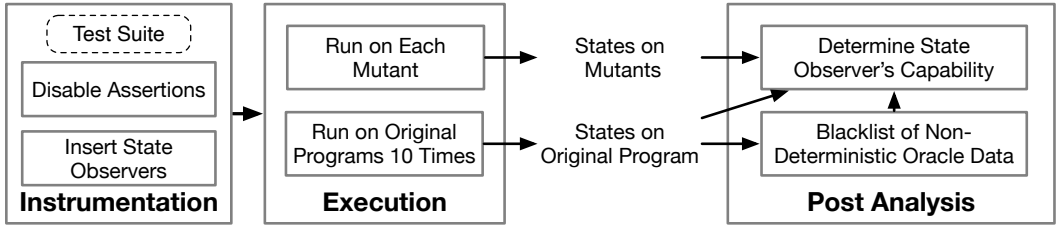


Fig. 3. Experimental Workflow

from existing research that studied the runtime effects of mutations [21]. Additionally, DUBBO-COMMON and INCUBATOR-SEATA-CORE are core modules of Apache projects that had the highest stars on GitHub on the date we conducted the experiment. MYBATIS is a widely-used Java framework essential for simplifying database interactions through SQL mapping.

In Table 1, we list the specifics of the subjects. The first seven columns in Table 1 list basic subject characteristics, including (1) subject project; (2) project version; (3) lines of production code; (4) number of tests; (5) line coverage; (6) number of covered mutants; and (7) mutation score for covered mutants.

4.2 Experimental Setup

We designed and implemented our experiment to run in a systematic workflow that we depict in Figure 3. We describe the steps in Figure 3 and explain our engineering choices when implementing this experiment.

Our experiment is implemented for the Java programming language. The goal of our experiment is to analyze how varying state-capture approaches detect faults in our subject programs. To achieve this goal, we source a large quantity of artificial code changes, *i.e.*, mutations, using PIT [15] (version 1.16.3) with all operators from its DEFAULT group, and employ mutation analysis to determine the sensitivity of each state-capture strategy. Next, we describe our memory-instrumentation framework that implements multiple state-capture approaches we outlined in Section 3.

Step 1: Instrumentation. We perform a two-step instrumentation for a given test suite: (a) disabling test assertions in test suites, and (b) inserting external state observers for test runs in a test suite.

First, we use automated bytecode manipulation to functionally disable test assertions from third-party testing libraries, such as JUNIT, ASSERTJ, and HAMCREST. We use the ASM bytecode manipulation library [12] to automatically rewrite these test-assertion calls at the bytecode level, redirecting them to wrapper methods that record each assertion's result but suppressing any assertion-failure exceptions. The majority of test assertions across our subjects come from JUNIT, with three subjects using occasional HAMCREST assertions and

three mixing `ASSERTJ` with `JUNIT`. After disabling, the tests in our subject programs fail only by non-assertion failures, *e.g.*, uncaught runtime exceptions.

Second, we further instrument the assertion-disabled test suites with program-state observers, using two types of probes: (1) oracle data sourcing probes, and (2) oracle data capture probes.

Oracle Data Sourcing Probe. Oracle data sourcing probes identify root variables that can be used to start the program-state walks from different program-state scopes used in a test assertion. As we describe in Section 3, such program-state scopes refer to the immediate stack and heap memory locations available within the test code to access:

- (1) **local** variables in test code, which we track with the `STORE` bytecode instruction in a test method's code;
- (2) **return** values stored transiently on the test method's operand stack, captured soon after `invoke` bytecode instructions: `INVOKEVIRTUAL`, `INVOKESTATIC`, `INVOKESPECIAL`, `INVOKEINTERFACE`, `NEWARRAY`, `ANEWARRAY`, and `MULTIANEWARRAY`;
- (3) **fields** on object instances, as accessed by `GETFIELD` or `GETSTATIC` bytecode instructions in a test method's code; and
- (4) publicly accessible **static** fields defined in the program.

Upon execution, an oracle data sourcing probe would store each such memory location — of various scope types — in a list that is accessible throughout the execution of the enclosing test method. For any test method, we refer to that list as the *state-scope list*.

Oracle Data Capture Probe. These probes execute once per each test method run, during the test method's exit or completion phase. As a test method proceeds with its execution, our oracle data sourcing probes would accumulate different state-scopes (*e.g.*, local variables, fields) onto the test method's state-scope list.

As a test method is completing its execution, the oracle data capture probe performs memory- and getter-walks, using the stack and heap memory locations sourced in the state-scope list as start points of such walks (see Figure 2 and Section 3). The memory- and getter-walks retrieve all accessible memory locations and their concrete values, while limiting all such walks up to state-extraction depth of 5. The probes capture and maintain the data from these getter-/memory-walks as memory graph data-structures that are rooted at the memory locations in the state-scope list, and store that data to disk.

We implement getter- and memory-walks in the capture probes using Java's reflection facilities. We use Java reflection to both access values stored in fields for memory-walks, and also for obtaining return values by invoking getter methods for getter-walks. We identify getter methods using the getter heuristics employed in `DSpor` [8]. During the oracle data capture process, we also track and identify parts of the program-state data that have already been captured to avoid duplication in the data stored on disk.

Additionally, we acknowledge that some techniques (*e.g.*, [27, 28])) use heuristics to produce assertions that check for equality relationships between two variables in the same test. To measure the power of such assertions, our oracle data capture probes also compute pairwise variable-equality results from memory locations in the state-scope list (using the `==` operator and the `equals` method, if available).

Step 2: Execution. We then run the instrumented test suites against both: (a) the original program (10 times); and (b) the mutants (once). We run the original program 10 times to guard against any non-deterministic program states in the original program. Note that test suites with disabled assertions can still fail on mutants, due to error checking in production code or uncaught exceptions [20]. Following prior works that evaluate assertions [37, 38, 48], we discarded mutants detected by non-test-assertions or uncaught exceptions. As a result, we collect all accessible program states on both mutants and the original program.

Step 3: Post Analysis. After executing mutants and the original program with the instrumented tests, we analyze the collected program states to determine state observers' sensitivity to detect erroneous program states triggered by mutations. We first compare program states on 10 original test runs to identify non-deterministic program states, and then identify detectable mutants by comparing program states between the original- and mutated-test runs.

The program-state comparison process consists of two phases. In the first phase, we identify test executions where the control flow of the test code diverges between the original and mutated runs. Such divergences

Table 2. Time cost for each experimental stage.

Phase	Total	Range (per subject)
Oracle collection (10×)	220 h	3–29 h
State comparison (original runs)	5.9 h	13 s–2.7 h
Mutation analysis (PIT)	19 h	3 m–6.5 h
Surviving mutant validation	62 h	20 m–26 h
Mutation runs w/ state instrumentation	414 h	1–89 h
State comparison	180 h	7.2 m–53 h

arise when the test prefix includes branching constructs (e.g., if-else, loops, or exception handling). Real-world human-written tests occasionally contain such logic, and when control flow diverges, subsequent state comparisons among specific parts of the prefix are unreliable and hence generally unusable. To detect these divergences, we analyze the trace of observed variables, *i.e.*, the sequence of comprehensively collected variables across the test prefix; inconsistencies between the original and mutated runs indicate a control-flow change specified in the test code. We therefore regard each diverging execution as invalid for fine-grained state comparison and exclude it from the subsequent analysis, with aggregated results reported in *RQ1-b*.

With the remaining test executions, we proceed with the second phase of program-state comparison: analyzing the state information of individual variables captured from different parts of the test prefix. Our oracle probes record program states as hierarchical memory graphs, where each node represents an object and edges represent fields or references. Each entry is annotated with the variable path, type, and value, allowing comparisons at varying depths of the object structure. Unlike naïve full serialization, our approach supports configurable depth bounds and avoids redundantly serializing attributes that have already been observed. This flexibility ensures that comparisons can range from shallow, surface-level fields to deep object graphs, depending on the configured state-extraction depth. The hierarchical structure is built through Java reflection, enabling traversal of complex user-defined types and container classes. Any detectable difference between the hierarchical graphs of the original and mutated runs provides an opportunity to synthesize assertions that would reveal behavioral changes, grounding the link between state-capture configuration (scope, approach, depth) and oracle effectiveness.

Experimental Cost. We report the experimental time for each stage of the experiment in Table 2. The full experiment ran for approximately six weeks on a 2021 MacBook Pro (Apple M1, 3.2 GHz, 16 GB RAM). The most expensive phase was running instrumented tests on each surviving mutant (414 h), followed by the repeated oracle collection for filtering non-deterministic state (220 h) and state comparison (180 h).

It is important to note that our implementation prioritizes comprehensive data collection for analyzing state-capture strategies rather than minimizing runtime. Several design choices in our framework inflate costs beyond what a practical tool would incur, including: (1) storing oracle data to disk rather than embedding it directly in test code; (2) repeating oracle collection ten times to filter non-deterministic state; (3) running each test in its own JVM for isolation; and (4) executing every surviving mutant with full state instrumentation. A practical tool would likely employ a subset of these measures, and its cost would further depend on factors such as optional mutation-based assertion minimization [27] and global time budgets that constrain assertion generation. As such, the times reported here reflect our controlled experimental setting and should not be interpreted as indicative of real-world tool costs.

5 Results

RQ1: How do program states, captured using different state-capture strategies, vary in their fault-revealing capabilities?

RQ1-a: State-Extraction Approaches. In Figure 4, we present a small-multiples [70] scatter plot comparing three state-extraction approaches, while controlling for observing scopes and extraction depths. Orange squares, blue dots, and green asterisks represent memory-walk, getter-walk, and getter-walk* (excluding hashCode and toString) approaches, respectively. The vertical axis indicates each approach’s ability to detect

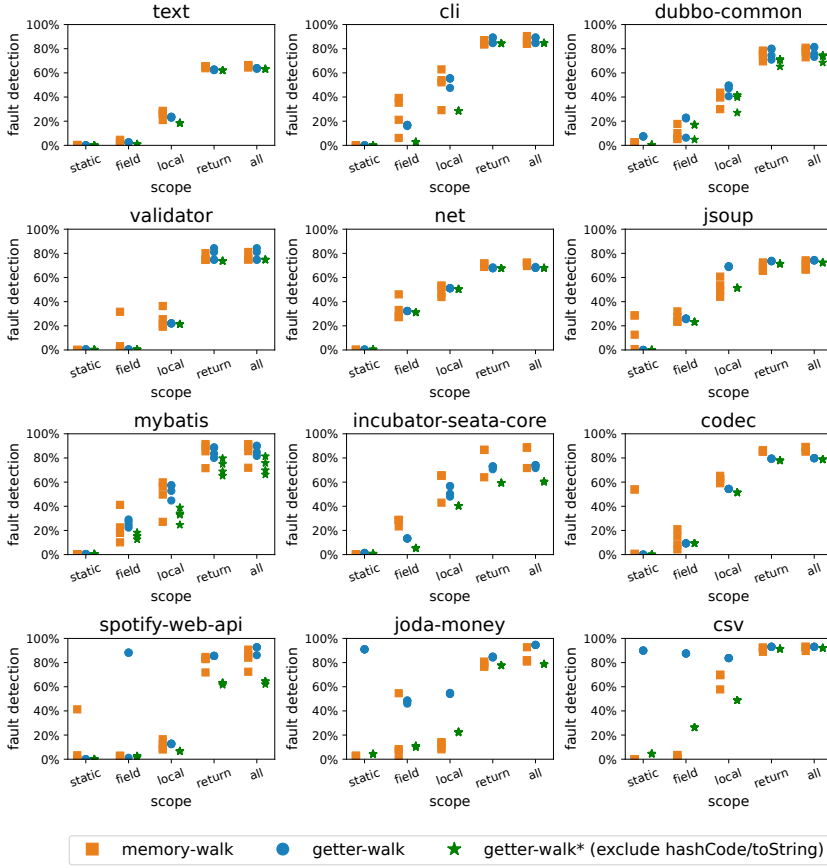


Fig. 4. Comparison of getter- and memory-walk state extraction approach

mutants, while the horizontal axis represents different state-observing scopes. For a given subject and scope, extraction depths from 2 to 5 appear as sets of orange squares and blue dots—greater depth corresponds to higher fault detection. Depth 1 is excluded, as getter- and memory-walk approaches yield identical results.

When comparing the orthodox getter-walk (including hashCode and toString) and memory-walk approaches, we found most projects show minimal differences when scope and depth are controlled, as indicated by similar heights of orange squares and blue dots. While minor variations exist, no consistent trend favors one approach over the other in fault-detection capabilities. For instance, in DUBBO-COMMON, getter-walk appears slightly superior, whereas in COMMONS-CODEC, memory-walk generally performs better across most depth configurations.

However, when excluding the toString and hashCode methods from the getter-walk approach, the assertions' fault detection capability drops indicated by the lowered green asterisks accompanying orange squares and blue dots. Such reductions have varied degrees across different subjects — by as much as 30.5% in SPOTIFY-WEB-API and as little as 1.1% in COMMONS-CSV with a depth of 5 and state observation across all scopes. On average, the fault-revealing capability decreases by 9.1% across studied subjects when excluding hashCode and toString from getter walks.

Observation 1-a: Getter-walk and memory-walk state-extraction approaches demonstrate comparable sensitivity in mutant detection; however, excluding toString and hashCode methods can reduce the capability of the getter-walk approach by 1.1%–30.4% across studied subjects.

Table 3. Percentages of Mutants Detected by Equality Assertions and Causing Divergent Test Logic

Subjects	Equality (%)	Divergence (%)
cli	15.3%	12.6%
csv	14.2%	10.4%
spotify-web-api	9.1%	4.3%
incubator-seata-core	12.7%	13.7%
validator	6.5%	22.4%
mybatis	19.0%	15.0%
text	15.3%	35.1%
joda-money	30.8%	5.2%
dubbo-common	17.7%	5.4%
net	7.6%	7.5%
codec	2.3%	13.9%
jsoup	11.5%	4.9%

RQ1-b: State-Observing Scope.

(1) *Specific Parts of the Test Prefix.* Figure 5 uses a series of line plots to illustrate how sensitivity to fault-detection varies across observing scopes as the state-extraction depth increases, specifically with the getter-walk* extraction approach. Each subplot represents a different scope, with depth (1–5) on the horizontal axis and the proportion of detected mutants on the vertical axis. Each line within a subplot corresponds to a project.

For state-observing scopes, we focus on depth level 5. Generally, observing **return** values were found more effective than **local** variables and **field** dereferences, with fault detection rates ranging from 59.3% to 91.3% (avg. 73.3%) for **return** values, 6.7% to 51.3% (avg. 35%) for **local** variables, and 0.4% to 31.2% (avg. 12.3%) for dereferenced **fields**. Observing states through **static** fields shows minimal impact on mutant detection, except in COMMONS-CSV and JODA-MONEY. Additionally, combining **all** scopes gives only a 1.6% increase in effectiveness compared to **return** values alone.

(2) *Equalities of objects across different parts of the test prefix.* We also investigated assertion strategies that check **object equalities** within the same test. These include identity checks (e.g., `assertSame`) or value-based checks (e.g., `assertEquals`). Such assertions are sometimes written by developers and commonly produced by test generation tools such as EvoSUITE [27]. The data are shown in the second column of Table 3. Overall, these strategies detected between 2.3% and 30.8% of mutants, with an average of 13.5%.

(3) *Divergence from branching statements in the test prefix.* Finally, we examined the effect of **branching statements** in the test prefix, such as *for* loops, *if-else* conditions, or *try-catch* blocks. These constructs can alter execution logic in ways that remain unnoticed without proper assertions. As shown in the third column in Table 3, across all subjects an average of 12.5% of mutants introduced such divergences, with rates reaching up to 35.1% in COMMONS-TEXT. This higher rate is explained by the presence of *for*-loop checks in COMMONS-TEXT, where faults can change the number of loop iterations silently, without being detected.

Observation 1-b: Observing return values outperforms other individual scopes and approximates the fault-revealing capability of observing program-state using all scopes. However, branching statements in the test prefix can introduce divergent execution paths, leaving to 35% of mutants undetected if without proper assertions.

RQ1-c: State-Extraction Depth. Figure 5 also illustrates how increasing depth in state walks increases assertions' sensitivity across various state-observing scopes. Generally, deeper state observation improves the sensitivity to detecting mutants. For example, increasing depth from 1 to 5 raises fault-detection rates on average from 65.5% to 81.5% for **return** values, 14.2% to 48.1% for **local** variables, and 2.8% to 26.9% for dereferenced **fields**. However, the sensitivity to depth varies by scope: observing **return** value is less depth-sensitive than **local** variables or **field** dereferences. Additionally, observing **static** fields only substantially impacts detection for three subjects—and only at depths beyond 3. We also notice that observing **return**

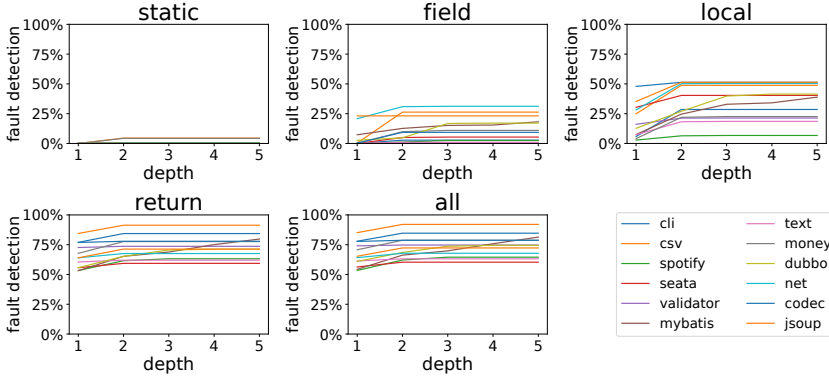


Fig. 5. Comparison of state-capture strategies that vary in depth and scope

values at depth 3 nearly matches the effectiveness of observing **all** variables at depth 5, with detection rates of 65.0%–92.0% (avg. 78.8%) compared to 66.5%–93.1% (avg. 84.3%).

Observation 1-c: By observing return values, increasing state-extraction depth from 1 to 5 can increase the state observers' fault-revealing capabilities by (on average) a factor of 24.4% in existing test suites. However, most of these gains occur by depth 3, with only marginal improvements beyond this point.

RQ2: How does the size of oracle data vary with state-extraction depths for each state-extraction approach? How does the size of oracle data vary with state-extraction depths for each state-extraction approach?

We quantify the oracle data size (defined in Section 2) for **return** values in each test case. Results from *RQ1* indicate that **return** values across all subjects provide substantial fault-detection capabilities — comparable to those of combining all state-observing scopes. Thus, we focus our cost analysis on **return** values.

Figure 6 presents oracle data sizes from three approaches at varying depths. For instance, in *SPOTIFY-WEB-API* at depth 5, the memory-walk state-extraction approach traverses, captures, and compares 13,305 granular data attributes as oracle data per test.

Both getter-walk extraction approaches consistently produce fewer oracle data beyond an observation depth of 1 than the memory-walk extraction approach. Including or excluding `toString` and `hashCode` methods has little impact on overall oracle data size. In contrast, memory-walk extraction causes a sharp increase in oracle data with depth, whereas getter-walk approaches grow more gradually. For example, in *JODA-MONEY*, memory-walk extraction increases oracle data size from 8.9 to 7,988.9 as depth rises from 1 to 5. This results in a 270× increase in the time required to write state to disk, a 161× increase in post-analysis time, and a 456× increase in on-disk storage size. By contrast, the getter-walk approach expands oracle data size to only 63.3 at depth 5, leading to a 2.5× increase in the time required to write state to disk, a 1.3× increase in post-analysis time, and a 2× increase in storage size.

Observation 2: Using the getter-walk extraction approach provides a more cost-effective way to observe state at varying depths (oracle data size growing up to 27× while increasing depth 1 to 5) as compared to the memory-walk extraction approach, which shows a sharp increase in oracle data size with extraction depth (oracle data size growing up to 897× while increasing depth 1 to 5).

RQ3: To what extent can observing varying program states amplify human-written tests?

In Figure 7, we compare the fault-detection capabilities (vertical axis) of state observers against existing assertions in each test suite across all subjects. Each subject has three lines for state observers using various state-extraction approaches. The red dotted line represents the baseline fault-detection capability of human-written assertions. Any data point above this line (shaded areas) indicates potential fault-detection

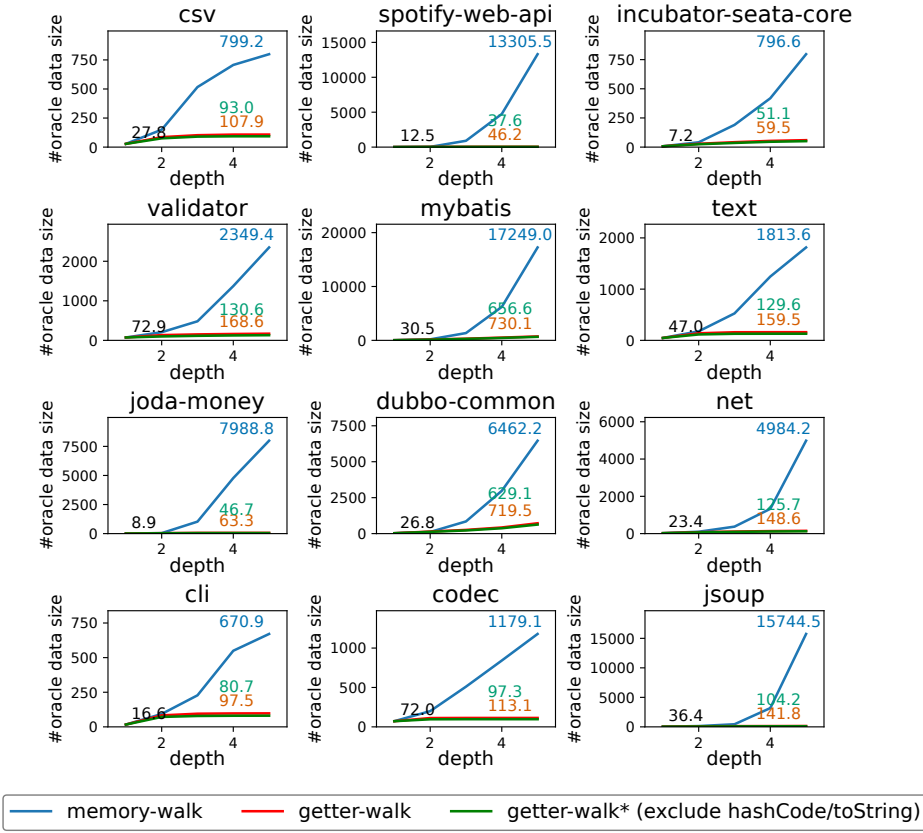


Fig. 6. The oracle data size being used per test case, using only return values as the state-observation scope.

improvements through amplified assertions. The unique contributions of `toString` and `hashCode` methods are highlighted in blue shades.

Our findings show that systematically asserting return values at depth 1 — using nullability, types, and primitive or string values — outperforms human-written assertions in fault detection for 7 out of 12 subjects. This highlights assertion-amplification opportunities through basic automated state observation. For example, in JSOUP, shallow (depth = 1) assertion amplification increases the fault-detection ratio from 44.6% (human assertions) to 65.7%, killing approximately 40% additional surviving mutants.

Deeper state observation generally enhances oracle’s sensitivity, often surpassing human assertions, as indicated by the overall shaded areas. However, an exception arises in COMMONS-CODEC: getter-walk state extraction at depth 5 fails to outperform human assertions, whereas memory-walk extraction approach does. Upon review, we found that 117 developer-written tests in COMMONS-CODEC use reflection to inspect object fields, limiting getter-walk extraction due to its reliance on available getter methods.

Additionally, some assertion enhancements may require `toString` and `hashCode` methods, as highlighted by blue areas. Their impact varies by subject: they can significantly improve fault detection in SPOTIFY-WEB-API, JODA-MONEY, INCUBATOR-SEATA-CORE, and MYBATIS but contribute less in other test suites. Overall, at depth 5, `toString` and `hashCode` account for an average of 37.3% of all amplification opportunities in human-written test suites.

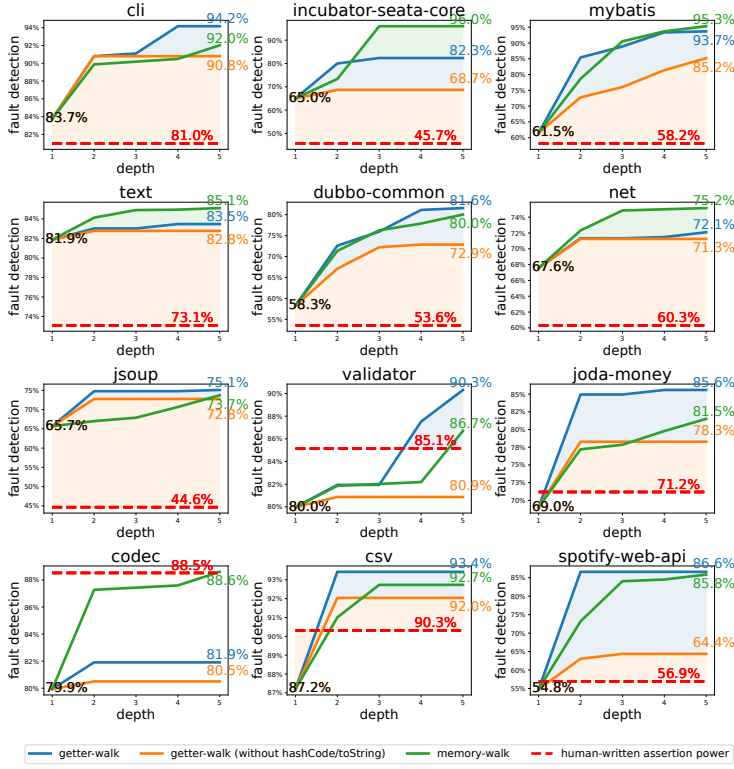


Fig. 7. Comparing state observer’s capability with human-written assertions already in the test suite. The shaded portions highlight opportunities to strengthen assertions in human-written test suites.

Observation 3: Assertion amplification can be conducted through expanding the observation depth and scope by using getters, which can improve existing assertion capability by a factor of 3.4% to 80.3% (avg. 35.7%) in 11 of the 12 subjects. Among them, on average 37.3% of contributions uniquely come from toString and hashCode methods, when using the getter approach.

6 Discussion and Implications

Through our analysis, we reported ranges of fault-revealing capabilities from different observable program states, offering insights into assertion generation and amplification. In this section, we extract all our insights and highlight some implications.

Effectiveness of Getter-walk Assertions: The Role of toString and hashCode. One might expect memory walks to be the most effective regression-detection strategy given their exhaustive nature, in contrast to getter-based approaches, which rely on the availability of accessors, as discussed in early work [73]. However, our analysis reveals that both getter- and memory-walk approaches can detect mutants with comparable capabilities; however, getter methods require significantly less oracle data. We further investigated getter methods and found that toString and hashCode provide unique contributions to mutant detection that other getters cannot replace. Excluding these methods results in a 37% reduction in potential assertion improvements in the tests we studied when getter methods alone are employed. These methods are disproportionately impactful because they typically aggregate an object’s entire state into a single return value—toString by concatenating significant fields into a string, hashCode by computing over multiple attributes—effectively

acting as compact memory-walk surrogates. This makes them highly sensitive to mutations, but also potentially more brittle under refactoring and more prone to nondeterminism (e.g., from collection iteration order). The impact varies across subjects: in SPOTIFY-WEB-API, excluding these methods causes a 30.5% drop, whereas in COMMONS-CSV the drop is only 1.1%, reflecting differences in object complexity and getter coverage.

However, the decision to incorporate `toString` and `hashCode` into assertions remains contentious. While these methods may produce verbose and brittle assertions, developers may intentionally include them to assert specific behaviors. In fact, we conducted a further investigation to see if developers typically include these methods in their test suites. We found *all* of our studied projects have test suites that utilize `toString` in some of their assertions, and 9 out of 12 projects use `hashCode` in assertions. In addition, automated techniques make decisions differently: EvoSUITE [27] excludes them, whereas DSPOT [8, 17] and RANDOOP [53] explicitly include them. However, most techniques do not clearly articulate their decisions regarding inclusion or exclusion, underscoring the need for explicit consideration of these methods in assertions.

Enhancing Program-State Observability in Testing. The unique contributions of the `toString` and `hashCode` methods also highlight limitations of encapsulation for testers, as these methods can capture object attributes not adequately exposed by existing getters. Consequently, testers and researchers might consider alternative strategies to appropriately enhance state observability for testing purposes, such as adding getter methods directly to production code, employing reflection to access object attributes, or developing specialized frameworks (e.g., [30]) to circumvent encapsulation restrictions. Indeed, Roslan *et al.* [56] found that developers frequently encounter visibility barriers when writing unit tests and employ various workarounds—including reflection, package-private access, and relaxing access modifiers—to observe otherwise inaccessible state. Their findings reinforce our observation that encapsulation limits the reach of getter-walk strategies and motivates deeper or alternative state-extraction approaches.

Impact of Test Prefix Structure and Control Flow. Our study highlights that the structure and elements of the test prefix have significant implications for oracle effectiveness. Prior work has similarly observed that testing tightly coupled classes often requires covering dependent components through method invocations, which increases test prefix size and complexity [62]. Unsurprisingly, we found that validating return values from the test prefix is both highly effective and a common practice; however, these values are often embedded in chained method calls and left unasserted in human-written tests. Our analysis reveals that making such return values explicit and asserting them sequentially can strengthen regression detection by, on average, more than 30%.

Moreover, the fact that depth-1 observations—which capture only nullability, types, and primitive or string return values—already outperform human-written assertions in 7 of 12 subjects (RQ3) suggests that many test suites lack basic checks on method-return values. For instance, in COMMONS-CLI, certain method-invocation returns are never asserted, allowing null-returning mutants to survive undetected. Even a simple null check or reusing the returned value in a subsequent assertion would catch such faults. This points toward lightweight analysis tools that can scan existing test suites and flag under-asserted return values as low-effort amplification candidates that achieves a higher test-suite effectiveness.

Additionally, the presence of conditional logic (e.g., `if-else`, loops, `try-catch`) within the test prefix can alter execution paths in subtle ways that may go unnoticed without appropriate assertions. Such constructs are sometimes regarded as test smells [3], since unit tests are generally expected to follow simple and linear execution patterns. Our empirical results support this view: divergences in test control flow accounted for an average of 12.5% of the mutants we studied, and in some projects reached as high as 35.1%.

This finding suggests that complex prefixes increase both the opportunities and risks of oracle construction. On one hand, they expose additional program states that may serve as valuable oracle targets. On the other, they complicate the mapping between input, prefix logic, and expected outcomes, potentially reducing the reliability of oracles. We argue that automated techniques should explicitly account for prefix complexity, either by avoiding overly complex control-flow logic in generated tests or by complementing such tests with assertions that guard against control-flow divergence (e.g., placing a *fail* assertion on specific branches or asserting the expected number of iterations in a loop). For practitioners, this highlights the importance of keeping test logic simple and straightforward, so that the relationship between test input, execution, and oracle remains transparent and maintainable.

Exploring Deeper Program States with Getters is an Affordable Approach to Improve Assertions' Capabilities. Current techniques typically rely on shallow observer methods (depth = 2) or perform full object serialization, resulting in assertions that are either limited in power or cluttered with bulky, brittle oracle data. However, we observed that utilizing deeper getter method chains to extract program states substantially enhances assertion power without substantially increasing oracle data size, while also potentially reducing the exposure of implementation details. For instance, `customer.getDiscountType().getDescription()` effectively retrieves a meaningful assertable attribute accessible from the `customer` variable, which would otherwise remain unreachable and as such non-assertable via a direct getter method call. Specifically, we found that adding just one additional getter step to access deeper program behaviors can increase fault detection by up to 7.7%, compared to relying solely on getters applied directly to prefix variables that return primitive values.

We further conducted a manual investigation on how assertions in human-written test suites assert variable attributes. We found that many tests access specific attributes through multiple chained getters (depth > 2) and/or retrieve values from collections, often requiring methods beyond standard getters—such as those accepting parameters to retrieve specific collection elements. For example, in COMMONS-CLI, over 80% of assertions follow such chained getter methods to assert variable object states.

Specialized Techniques Can Expand the State-Observation Scope and Depth to Further Improve Assertions' Capabilities. While asserting only return values often contributes most to mutant detection, expanding the observation scope and depth within the three-dimensions of the state-capturing strategies can further enhance assertions by revealing additional faults. Such expansions in state-observing are particularly beneficial in specialized scenarios, including resolving stubborn mutants or amplifying existing tests. Examples include DSpot [8], the first test amplification tool on human-written tests that can extend assertions by invoking non-recursive getters, while it includes `toString` and `hashCode` methods as getter methods. Another recent expansion attempt includes improving machine-generated tests' (i.e., EvoSuite) observability by bypassing the encapsulation mechanism [30].

Assertion Sensitivity: Brittleness or Fault-Detection Capability. In our experiments, we use mutations to assess assertions' sensitivity in detecting faults. This sensitivity reflects assertions' fault-revealing capabilities, a common practice in assertion evaluation. The underlying assumption is that mutations serve as proxies for faults that test suites aim to detect. However, high sensitivity may also reflect higher potentials to brittleness — when assertions in test cases are overly specific or fragile, breaking easily with minor system changes (just like mutations), even if those changes are functionally correct.

It remains an open question how different assertion types perform or complement each other. Potential mitigation for brittleness is selecting a targeted oracle dataset, such as using getter-walk approaches, while the memory-walk method (e.g., full object serialization) is also employed in industrial settings [4] (e.g., for the Instagram program) and in recent research [68, 71]. Additionally, mutation-based techniques [27] can reduce oracle data size while maintaining fault-detection sensitivity, thereby lowering the inevitable human-verification cost. However, snapshot-based approaches store full object information or memory snapshot of a test in an assertion rather than delegating into multiple individual assertions checking specific attributes, making minimization approaches less effective.

Relatedly, the relationship between state-capture strategies and test flakiness remains underexplored. While brittleness and flakiness are related concerns, we distinguish them here: brittleness refers to overly sensitive assertions that break under legitimate code changes, whereas flakiness refers to non-deterministic test outcomes across repeated executions of unchanged code [14, 33, 50, 58]. Our experimental design mitigates some non-determinism by running tests ten times, filtering inconsistent state, and executing each test in its own JVM. However, ten runs may not surface all non-deterministic behaviors, and our setup does not account for order-dependent flakiness. In future work, we plan to investigate how state-capture depth, scope, and extraction approach influence flakiness and brittleness in practice, and to explore mitigation techniques such as NonDex [34].

Assertions' Desirability. Our study did not conduct human studies to directly assess developer preferences regarding assertions produced by different techniques. In our work, we specifically discussed the usage of `toString` and `hashCode` in automated techniques and human-written tests, and their capability of revealing faults. However, developer preferences regarding the choice of these methods — or alternative approaches

— remain uncertain. As such, preferences may differ based on the development context and specific project requirements. For instance, assertions with serialized string oracle data might offer strong fault detection but result in verbose, hard-to-read oracles, which may complicate maintenance and debugging. Recent research has started addressing some of these desirability issues. Meta’s approach [4], for example, involves storing comprehensive memory snapshots of objects as oracle data. Although these snapshots are bulky, they significantly enhance the debugging experience by precisely pinpointing erroneous attributes within complex objects upon test failures, thereby improving the interpretability of assertion failure information. Additionally, ProDJ [71] serializes Java runtime objects into readable code, providing a clear alternative to XML or string representations. More broadly, developer satisfaction with amplified tests extends beyond oracle readability to factors such as test understandability, perceived usefulness, and willingness to maintain the added assertions. Brandt and Zaidman [10] found that developers often reject amplified tests not because of fault-detection deficiencies but due to poor readability, redundancy, or misalignment with their testing intent. These findings suggest that state-capture strategies should be evaluated not only by mutation score but also by the maintainability and interpretability of the resulting assertions—dimensions our current study does not measure. In future work, we will conduct human studies to evaluate assertion desirability and explore intelligent assertion selection, minimization, and prioritization techniques.

Traditional State Observation vs. LLM-Based Assertions. While our background section outlined assertion techniques that derive from external descriptions or LLM inference, here we reflect on their relationship to traditional state-observation-based methods. The two belong to fundamentally different categories: state observation yields assertions anchored in actual runtime behavior, whereas LLM-driven or intent-based techniques prioritize developer-aligned expectations. As such, their trade-offs are orthogonal—coverage and regression strength on one hand, readability and human alignment on the other.

At the same time, early empirical evidence suggests that LLM-based approaches alone do not yet match the regression effectiveness of traditional methods (e.g., TOGLL’s reported 17% gap compared to EvoSUITE). The landscape therefore points toward complementary integration: LLMs may guide variable selection, propose intent-aligned checks, or improve readability, while systematic observation ensures regression strength and completeness. We see future progress not in choosing one family of approaches over the other, but in combining their strengths to balance coverage, usability, and trustworthiness.

7 Related Work

Earlier research leveraged internal program execution data [35, 36, 74] as oracles, often relying on internal execution information—such as execution paths, branch conditions, or variable states within production code—making them impractical for many real-world scenarios. And we elaborate in Section 2.1 on how varied state capture strategies [4, 8, 17, 27–29, 32, 49, 53, 57, 63–65, 68, 71, 73, 75] differ in capturing, processing, and using program behaviors as assertion oracles. These idiosyncratic design choices are embedded in various test-generation techniques, including pure regression-assertion generation (e.g., ORSTRA [73]), human-in-the-loop assertion generation (e.g., AUTOASSERT [75]), full test-suite generation (e.g., EvoSUITE [27], RANDOOP [53], AGITARONE [63]), test amplification (e.g., DSPOT [8, 17], its developer-facing IDE integration TEST-CUBE [10], and EvoSUITE_{Amp} [55]), program assertions (e.g., GASSERT [64, 65], DAIKON [24]), and pre-conditioning checkers [6].

While the heuristic-based state-extraction approaches are demonstrated and explained in Section 2.1 and Section 3, the full-serialization, snapshot-based state observation strategies have interesting variations. For example, Tiwari *et al.* developed RICK [68] to extract test cases from production execution, in which oracles are produced through serializing and deserializing previously captured program states using [1]. Interestingly, a conceptually similar idea emerged in industry under the name *snapshot testing*, first proposed for UI testing (e.g., [32]) and later extended to program state reachable from object references. For example, Alshahwan *et al.* [4] developed a serialization framework to capture Java object states for regression testing of the Instagram application. The oracle data in the above-mentioned works are usually stored on disk and loaded as the expected value in the test assertions. Other variants of serialization-based oracles have likewise been designed by researchers, leading to diverse strategies for embedding object state into test assertions. For example, AUTOASSERT [75] have test assertions that hard-code the object’s serialized string values. GASSERT [64, 65] use a customized serialization approach to generate assertions: they extract all transitively reachable primitive

fields and primitive return values from getter methods, then serialize them into strings. Recently, Wachter *et al.* [71] proposed plain-code serialization, a form of serialization-based oracle that reconstructs Java objects directly as executable source code. Taken together, these idiosyncratic serialization strategies also reflect distinct considerations regarding the readability, understandability, and practical desirability of oracle data that deserve further investigation.

Nevertheless, these unique design considerations fundamentally embody different approaches to fault-propagation analysis, leveraging assertions to detect faults. The theoretical foundation for this process is encapsulated by the RIPR model [5, 44, 45], emphasizing that effective tests must incorporate appropriate oracles capable of identifying erroneous states resulting from fault execution. Our work specifically investigated where infection propagation is accessible through the test suite and how it can be accessed using different state-observing strategies. Moreover, while prior model-based testing research [11, 44, 45] has examined assertion-writing strategies within constrained timeframes and simple object-oriented contexts—employing models (e.g., finite state machines) and modeling languages (e.g., Java Modeling Language [13])—our work focuses on general regression assertion-generation strategies. We propose a conceptual framework supported by automation to systematically investigate the diverse state-capturing strategies and the resultant assertions. Additionally, we introduce the concept of *oracle data size*, which quantifies the informational content used within assertion oracles, independent of the specific assertion format (e.g., verifying serialized string representations or individual attributes of objects).

Concurrently, Molina *et al.* [52] proposed *state field coverage*, a metric that measures the proportion of an object’s class fields that an oracle may access during test execution. They found that state field coverage strongly correlates with fault-detection ability, reinforcing our observation that the breadth and depth of state captured by an assertion influences its effectiveness. And recently, Galindo-Gutiérrez *et al.* [30] improved EvoSUITE by transforming the code under test to expose encapsulated attributes and extending EvoSuite’s assertion capabilities for complex objects. Their work demonstrates that improving observability directly increases the fault-detection effectiveness of generated tests—a finding that aligns with our results showing that deeper state extraction and broader observation scope improve assertion sensitivity. In contrast, our framework characterizes the whole design space of state-capture strategies along three complementary dimensions that together determine which fields are reachable by an oracle, and we conduct empirical studies within the framework in fine-grained granularities.

8 Threats to Validity

Experimental Limitations. First, we use PIT [15] as a mutation-testing tool to evaluate test suites and their assertions, which may yield different empirical results compared to other mutation-testing tools, such as MuJAVA [51] and MAJOR [43]. However, PIT is widely used in research [54], industry, and open-source projects, and we utilized its default group of mutation operators. Moreover, unlike source-level mutation tools, PIT operates on Java bytecode, meaning that its mutation operators reflect bytecode-level transformations. As a result, certain source-level mutation patterns may not be represented, and some bytecode-specific mutations may not have direct source-level counterparts, which could influence the generalizability of our results to studies using source-level mutation tools. Additionally, PIT’s DEFAULT operators generate only first-order mutants, each introducing a single syntactic change (at the bytecode level). Higher-order mutants [40], which combine multiple changes, may exhibit different fault-propagation behaviors and could affect the relative effectiveness of state-capture strategies. Finally, while mutations are widely used as proxies for real faults [42, 54], they may not fully represent the complexity of real-world bugs, which can span multiple statements, methods, or even files.

Second, we studied 12 software subjects with human-written tests. Most of these subjects are drawn from DEFECTS4J [41], previous research on mutation effects [21], and we additionally included two open-source Apache projects with the highest star counts. Moreover, our analysis does not consider all types of assertions that may appear in human-written test suites, such as assertions that check relations. However, our analysis framework covers assertions checking concrete values with a range of specificity. Moreover, we also investigated the *equality* assertion produced by multiple automated tools [27, 28].

Generalizability. Our findings on various state-capture strategies for assertion generation may not generalize to other object-oriented languages. For instance, previous research on JavaScript found that tests typically contain a single, straightforward assertion [75], while our analysis of Java tests shows multiple assertions. Nevertheless, our conceptual framework, including state-observation scope, approach, and depth, may apply to other object-oriented programming languages, as the fundamental concepts of fault infection, error propagation, and detectability persist across programming languages.

More broadly, our framework and implementation are tailored to Java programs, where state is accessed through object references, fields, and getter methods. The conceptual dimensions of our framework—observation scope, extraction approach, and extraction depth—are language-agnostic and applicable to any language with structured, heap-allocated state. However, several implementation aspects would require adaptation for other languages. For example, adapting our approach to Python would require handling its dynamic typing, runtime attribute resolution, and lack of access modifiers, which affect how fields and methods are discovered and traversed. Languages that are not primarily object-oriented would require rethinking the extraction-approach dimension, as getter methods and field access may not be the dominant means of exposing program state. Similarly, our current scope of “program state” is limited to in-memory, heap-resident data accessible from test variables. Programs whose primary behavior is expressed through external side effects—such as CLI tools producing console output, GUI applications rendering visual state, or services writing to databases—would require additional mechanisms to capture and compare those external observations alongside in-memory state.

Clean Program Assumption. The state-capturing strategies discussed and studied in this paper for regression assertions assume that the program under test is correct, which derives oracles from the program’s execution properties as expected behaviors [7]. Assertions that do not rely on execution are limited by the resources from documentation and/or knowledge learned by machine learning models. They typically generate only a few assertions [19, 37, 72] revealing fewer faults due to their limited assertable space. Specifically, the LLM-based technique, TOGLL [37] in 2024, kills 17% fewer mutants than EvoSuite-generated tests.

9 Conclusion

In this work, we examined the role of state-capture strategies in determining how effectively test oracles reveal regressions. Based on this, we introduced a conceptual framework that characterizes design choices along three dimensions for capturing memory state for assertion generation. Using our framework, we conducted an experiment to investigate the impact of capturing diverse memory states in test code. The results of our study reveal useful guidance in designing automated test assertion generation techniques. Our analysis further exposes the state observability and assertion desirability challenges faced by automated test-generation tool builders and researchers. In future work, we aim to extend these studies to a broader range of subject programs, programming languages, and test environments, as well as conduct user studies to evaluate the relative desirability of generated assertions across different approaches.

Acknowledgments

The second author’s opinions expressed in this publication are solely his and do not purport to reflect the opinions or views of his employer, Microsoft.

10 Data Availability

We have made the source code and data used in this study publicly accessible. They are available at [2], which contains necessary files and documentation for replicating our results.

References

- [1] 2022. XStream - About XStream. <https://x-stream.github.io/> [Accessed 01-08-2023].
- [2] 2026. Replication Artifact: GitHub AssertionStrategyAnalysis. <https://github.com/spideruci/AssertionStrategyAnalysis> [Accessed 04-23-2026].
- [3] Wajdi Aljedaani, Anthony Peruma, Ahmed Aljohani, Mazen Alotaibi, Mohamed Wiem Mkaouer, Ali Ouni, Christian D. Newman, Abdullatif Ghallab, and Stephanie Ludi. 2021. Test Smell Detection Tools: A Systematic Mapping Study. In

- Evaluation and Assessment in Software Engineering* (Trondheim, Norway) (EASE 2021). Association for Computing Machinery, New York, NY, USA, 170–180. <https://doi.org/10.1145/3463274.3463335>
- [4] Nadia Alshahwan, Mark Harman, Alexandru Marginean, Rotem Tal, and Eddy Wang. 2024. Observation-Based Unit Test Generation at Meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (Porto de Galinhas, Brazil) (FSE 2024). Association for Computing Machinery, New York, NY, USA, 173–184. <https://doi.org/10.1145/3663529.3663838>
 - [5] Paul Ammann and Jeff Offutt. 2016. *Introduction to software testing*. Cambridge University Press.
 - [6] James H. Andrews, Felix C. H. Li, and Tim Menzies. 2007. Nighthawk: a two-level genetic-random unit test data generator. In *Proceedings of the 22nd IEEE/ACM International Conference on Automated Software Engineering* (Atlanta, Georgia, USA) (ASE '07). Association for Computing Machinery, New York, NY, USA, 144–153. <https://doi.org/10.1145/1321631.1321654>
 - [7] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering* 41, 5 (2015), 507–525. <https://doi.org/10.1109/TSE.2014.2372785>
 - [8] Benoit Baudry, Simon Allier, Marcelino Rodriguez-Cancio, and Martin Monperrus. 2015. DSpot: Test Amplification for Automatic Assessment of Computational Diversity. arXiv:1503.05807 [cs.SE] <https://arxiv.org/abs/1503.05807>
 - [9] Carolin Brandt, Ali Khatami, Mairieli Wessel, and Andy Zaidman. 2024. Shaken, Not Stirred: How Developers Like Their Amplified Tests. *IEEE Transactions on Software Engineering* 50, 5 (2024), 1264–1280. <https://doi.org/10.1109/TSE.2024.3381015>
 - [10] Carolin Brandt and Andy Zaidman. 2022. Developer-centric test amplification: The interplay between automatic generation human exploration. *Empirical Softw. Engg.* 27, 4 (July 2022), 35 pages. <https://doi.org/10.1007/s10664-021-10094-2>
 - [11] L.C. Briand, M. Di Penta, and Y. Labiche. 2004. Assessing and improving state-based class testing: a series of experiments. *IEEE Transactions on Software Engineering* 30, 11 (2004), 770–783. <https://doi.org/10.1109/TSE.2004.79>
 - [12] E. Bruneton, R. Lenglet, and T. Coupaye. 2002. ASM: A code manipulation tool to implement adaptable systems. In *Adaptable and Extensible Component Systems*.
 - [13] Lilian Burdy, Yoonsik Cheon, David Cok, Michael D. Ernst, Joe Kiniry, Gary T. Leavens, K. Rustan, M. Leino, and Erik Poll. 2003. An overview of JML tools and applications1 www.jmlspecs.org. *Electronic Notes in Theoretical Computer Science* 80 (2003), 75–91. [https://doi.org/10.1016/S1571-0661\(04\)80810-7](https://doi.org/10.1016/S1571-0661(04)80810-7) Eighth International Workshop on Formal Methods for Industrial Critical Systems (FMICS'03).
 - [14] Yang Chen, Alperen Yildiz, Darko Marinov, and Reyhaneh Jabbarvand. 2023. Transforming Test Suites into Croissants. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) (ISSTA 2023). Association for Computing Machinery, New York, NY, USA, 1080–1092. <https://doi.org/10.1145/3597926.3598119>
 - [15] Henry Coles, Thomas Laurent, Christopher Henard, Mike Papadakis, and Anthony Ventresque. 2016. Pit: a practical mutation testing tool for java. In *Proceedings of the 25th international symposium on software testing and analysis*. 449–452. <https://doi.org/10.1145/2931037.2948707>
 - [16] Benjamin Danglot, Oscar Vera-Perez, Zhongxing Yu, Andy Zaidman, Martin Monperrus, and Benoit Baudry. 2019. A snowballing literature study on test amplification. *Journal of Systems and Software* 157 (2019), 110398. <https://doi.org/10.1016/j.jss.2019.110398>
 - [17] Benjamin Danglot, Oscar Luis Vera-Pérez, Benoit Baudry, and Martin Monperrus. 2019. Automatic test improvement with DSpot: a study with ten mature open-source projects. *Empirical Software Engineering* 24, 4 (2019), 2603–2635. <https://doi.org/10.1007/s10664-019-09692-y>
 - [18] Amirhossein Deljouyi, Roham Koohestani, Maliheh Izadi, and Andy Zaidman. 2025. Leveraging Large Language Models for Enhancing the Understandability of Generated Unit Tests. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 1449–1461. <https://doi.org/10.1109/ICSE55347.2025.00032>
 - [19] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K. Lahiri. 2022. TOGA: a neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) (ICSE '22). Association for Computing Machinery, New York, NY, USA, 2130–2141. <https://doi.org/10.1145/3510003.3510141>
 - [20] Hang Du, Vijay Krishna Palepu, and James A. Jones. 2023. To Kill a Mutant: An Empirical Study of Mutation Testing Kills. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) (ISSTA 2023). Association for Computing Machinery, New York, NY, USA, 715–726. <https://doi.org/10.1145/3597926.3598090>
 - [21] Hang Du, Vijay Krishna Palepu, and James A. Jones. 2024. Ripples of a Mutation — An Empirical Study of Propagation Effects in Mutation Testing. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) (ICSE '24). Association for Computing Machinery, New York, NY, USA, Article 116, 13 pages.

- <https://doi.org/10.1145/3597503.3639179>
- [22] Hang Du, Vijay Krishna Palepu, and James A. Jones. 2025. Leveraging Propagated Infection to Crossfire Mutants . In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 3136–3148. <https://doi.org/10.1109/ICSE55347.2025.00150>
 - [23] Sebastian Elbaum, Hui Nee Chin, Matthew B. Dwyer, and Matthew Jorde. 2009. Carving and Replaying Differential Unit Test Cases from System Test Cases. *IEEE Trans. Softw. Eng.* 35, 1 (Jan. 2009), 29–45. <https://doi.org/10.1109/TSE.2008.103>
 - [24] M.D. Ernst, J. Cockrell, W.G. Griswold, and D. Notkin. 2001. Dynamically discovering likely program invariants to support program evolution. *IEEE Transactions on Software Engineering* 27, 2 (2001), 99–123. <https://doi.org/10.1109/32.908957>
 - [25] Robert B. Evans and Alberto Savoia. 2007. Differential testing: a new approach to change detection. In *The 6th Joint Meeting on European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering: Companion Papers* (Dubrovnik, Croatia) (*ESEC-FSE companion '07*). Association for Computing Machinery, New York, NY, USA, 549–552. <https://doi.org/10.1145/1295014.1295038>
 - [26] Mattia Fazzini, Chase Choi, Juan Manuel Copia, Gabriel Lee, Yoshiki Kakehi, Alessandra Gorla, and Alessandro Orso. 2022. Use of test doubles in Android testing: an in-depth investigation. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) (*ICSE '22*). Association for Computing Machinery, New York, NY, USA, 2266–2278. <https://doi.org/10.1145/3510003.3510175>
 - [27] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering* (Szeged, Hungary) (*ESEC/FSE '11*). Association for Computing Machinery, New York, NY, USA, 416–419. <https://doi.org/10.1145/2025113.2025179>
 - [28] Gordon Fraser and Andrea Arcuri. 2013. Whole Test Suite Generation. *IEEE Transactions on Software Engineering* 39, 2 (2013), 276–291. <https://doi.org/10.1109/TSE.2012.14>
 - [29] Gordon Fraser and Andreas Zeller. 2010. Mutation-driven generation of unit tests and oracles. In *Proceedings of the 19th international symposium on Software testing and analysis*. 147–158. <https://doi.org/10.1145/1831708.1831728>
 - [30] Geraldine Galindo-Gutierrez, Juan Pablo Sandoval Alcocer, Nicolas Jimenez-Fuentes, Alexandre Bergel, and Gordon Fraser. 2025. Increasing the Effectiveness of Automatically Generated Tests by Improving Class Observability. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 1553–1565. <https://doi.org/10.1109/ICSE55347.2025.00156>
 - [31] Gregory Gay, Matt Staats, Michael Whalen, and Mats P. E. Heimdahl. 2015. Automated Oracle Data Selection Support. *IEEE Transactions on Software Engineering* 41, 11 (2015), 1119–1137. <https://doi.org/10.1109/TSE.2015.2436920>
 - [32] Victor Pezzi Gazzinelli Cruz, Henrique Rocha, and Marco Tulio Valente. 2023. Snapshot testing in practice: Benefits and drawbacks. *Journal of Systems and Software* 204 (2023), 111797. <https://doi.org/10.1016/j.jss.2023.111797>
 - [33] Martin Gruber, Muhammad Firhard Roslan, Owain Parry, Fabian Scharnböck, Phil McMin, and Gordon Fraser. 2024. Do Automatic Test Generation Tools Generate Flaky Tests?. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) (*ICSE '24*). Association for Computing Machinery, New York, NY, USA, Article 47, 12 pages. <https://doi.org/10.1145/3597503.3608138>
 - [34] Alex Gyori, Ben Lambeth, August Shi, Owolabi Legunsen, and Darko Marinov. 2016. NonDex: a tool for detecting and debugging wrong assumptions on Java API specifications. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Seattle, WA, USA) (*FSE 2016*). Association for Computing Machinery, New York, NY, USA, 993–997. <https://doi.org/10.1145/2950290.2983932>
 - [35] Mary Jean Harrold, Gregg Rothermel, Kent Sayre, Rui Wu, and Liu Yi. 2000. An empirical investigation of the relationship between spectra differences and regression faults. *Software Testing, Verification and Reliability* 10, 3 (2000), 171–194. [https://doi.org/10.1002/1099-1689\(200009\)10:3<171::AID-STVR209>3.0.CO;2-J](https://doi.org/10.1002/1099-1689(200009)10:3<171::AID-STVR209>3.0.CO;2-J)
 - [36] Mary Jean Harrold, Gregg Rothermel, Rui Wu, and Liu Yi. 1998. An empirical investigation of program spectra. In *Proceedings of the 1998 ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering* (Montreal, Quebec, Canada) (*PASTE '98*). Association for Computing Machinery, New York, NY, USA, 83–90. <https://doi.org/10.1145/277631.277647>
 - [37] Soneya Binta Hossain and Matthew Dwyer. 2024. TOGLL: Correct and Strong Test Oracle Generation with LLMs. arXiv:2405.03786 [cs.SE] <https://arxiv.org/abs/2405.03786>
 - [38] Soneya Binta Hossain, Antonio Filiari, Matthew B. Dwyer, Sebastian Elbaum, and Willem Visser. 2023. Neural-Based Test Oracle Generation: A Large-Scale Evaluation and Lessons Learned. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) (*ESEC/FSE 2023*). Association for Computing Machinery, New York, NY, USA, 120–132. <https://doi.org/10.1145/3611643.3616265>
 - [39] Hojun Jaygarl, Sunghun Kim, Tao Xie, and Carl K. Chang. 2010. OCAT: object capture-based automated testing. In *Proceedings of the 19th International Symposium on Software Testing and Analysis* (Trento, Italy) (*ISSTA '10*). Association

- for Computing Machinery, New York, NY, USA, 159–170. <https://doi.org/10.1145/1831708.1831729>
- [40] Yue Jia and Mark Harman. 2009. Higher Order Mutation Testing. *Information and Software Technology* 51, 10 (2009), 1379–1393. <https://doi.org/10.1016/j.infsof.2009.04.016> Source Code Analysis and Manipulation, SCAM 2008.
- [41] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis* (San Jose, CA, USA) (*ISSTA 2014*). Association for Computing Machinery, New York, NY, USA, 437–440. <https://doi.org/10.1145/2610384.2628055>
- [42] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. 2014. Are mutants a valid substitute for real faults in software testing?. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Hong Kong, China) (*FSE 2014*). Association for Computing Machinery, New York, NY, USA, 654–665. <https://doi.org/10.1145/2635868.2635929>
- [43] René Just, Franz Schwegert, and Gregory M. Kapfhammer. 2011. MAJOR: An efficient and extensible tool for mutation analysis in a Java compiler. In *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*. 612–615. <https://doi.org/10.1109/ASE.2011.6100138>
- [44] Nan Li and Jeff Offutt. 2014. An Empirical Analysis of Test Oracle Strategies for Model-Based Testing. In *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation*. 363–372. <https://doi.org/10.1109/ICST.2014.49>
- [45] Nan Li and Jeff Offutt. 2017. Test Oracle Strategies for Model-Based Testing. *IEEE Transactions on Software Engineering* 43, 4 (2017), 372–395. <https://doi.org/10.1109/TSE.2016.2597136>
- [46] Igor Lima, Jefferson Silva, Breno Miranda, Gustavo Pinto, and Marcelo d’Amorim. 2021. Exposing bugs in JavaScript engines through test transplantation and differential testing. *Software Quality Journal* 29, 1 (March 2021), 129–158. <https://doi.org/10.1007/s11219-020-09537-8>
- [47] Yu Liu, Pengyu Nie, Anna Guo, Milos Gligoric, and Owolabi Legunsen. 2023. Extracting Inline Tests from Unit Tests. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) (*ISSTA 2023*). Association for Computing Machinery, New York, NY, USA, 1458–1470. <https://doi.org/10.1145/3597926.3598149>
- [48] Zhongxin Liu, Kui Liu, Xin Xia, and Xiaohu Yang. 2023. Towards More Realistic Evaluation for Neural Test Oracle Generation. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) (*ISSTA 2023*). Association for Computing Machinery, New York, NY, USA, 589–600. <https://doi.org/10.1145/3597926.3598080>
- [49] Pablo Loyola, Matt Staats, In-Young Ko, and Gregg Rothermel. 2014. Dodona: automated oracle data set selection. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis* (San Jose, CA, USA) (*ISSTA 2014*). Association for Computing Machinery, New York, NY, USA, 193–203. <https://doi.org/10.1145/2610384.2610408>
- [50] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Hong Kong, China) (*FSE 2014*). Association for Computing Machinery, New York, NY, USA, 643–653. <https://doi.org/10.1145/2635868.2635920>
- [51] Yu-Seung Ma, Jeff Offutt, and Yong Rae Kwon. 2005. MuJava: an automated class mutation system: Research Articles. *Softw. Test. Verif. Reliab.* 15, 2 (June 2005), 97–133. <https://doi.org/10.1002/stvr.308>
- [52] Facundo Molina, Nazareno Aguirre, and Alessandra Gorla. 2025. State Field Coverage: A Metric for Oracle Quality. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (Seoul, Korea, Republic of). IEEE Press, 2707–2719. <https://doi.org/10.1109/ASE63991.2025.00222>
- [53] Carlos Pacheco and Michael D. Ernst. 2007. Randoop: feedback-directed random testing for Java. In *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion* (Montreal, Quebec, Canada) (*OOPSLA ’07*). Association for Computing Machinery, New York, NY, USA, 815–816. <https://doi.org/10.1145/1297846.1297902>
- [54] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. 2019. Chapter Six - Mutation Testing Advances: An Analysis and Survey. *Advances in Computers*, Vol. 112. Elsevier, 275–378. <https://doi.org/10.1016/bs.adcom.2018.03.015>
- [55] Muhammad Firhard Roslan, José Miguel Rojas, and Phil McMinn. 2022. An Empirical Comparison of EvoSuite and DSpot for Improving Developer-Written Test Suites with Respect to Mutation Score. In *Search-Based Software Engineering: 14th International Symposium, SSBSE 2022, Singapore, November 17–18, 2022, Proceedings* (Singapore, Singapore). Springer-Verlag, Berlin, Heidelberg, 19–34. https://doi.org/10.1007/978-3-031-21251-2_2
- [56] Muhammad Firhard Roslan, José Miguel Rojas, and Phil McMinn. 2024. Private-Keep Out? Understanding How Developers Account for Code Visibility in Unit Testing. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 312–324. <https://doi.org/10.1109/ICSME58944.2024.00037>

- [57] Sina Shamshiri, René Just, José Miguel Rojas, Gordon Fraser, Phil McMinn, and Andrea Arcuri. 2015. Do Automatically Generated Unit Tests Find Real Faults? An Empirical Study of Effectiveness and Challenges (T). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 201–211. <https://doi.org/10.1109/ASE.2015.86>
- [58] August Shi, Jonathan Bell, and Darko Marinov. 2019. Mitigating the effects of flaky tests on mutation testing. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 112–122. <https://doi.org/10.1145/3293882.3330568>
- [59] Francisco Carlos Souza, M Souza, Mike Papadakis, Vinicius Durelli, and Márcio Delamaro. 2014. Test Data Generation Techniques for Mutation Testing: A Systematic Mapping. <https://doi.org/10.13140/RG.2.1.3699.9209>
- [60] Matt Staats, Pablo Loyola, and Gregg Rothermel. 2012. Oracle-Centric Test Case Prioritization. In *2012 IEEE 23rd International Symposium on Software Reliability Engineering*. 311–320. <https://doi.org/10.1109/ISSRE.2012.13>
- [61] John Steven, Pravir Chandra, Bob Fleck, and Andy Podgurski. 2000. jRapture: A Capture/Replay tool for observation-based testing. *SIGSOFT Softw. Eng. Notes* 25, 5 (Aug. 2000), 158–167. <https://doi.org/10.1145/347636.348993>
- [62] Amjed Tahir. 2016. *A study on software testability and the quality of testing in object-oriented systems*. Ph. D. Dissertation. University of Otago.
- [63] Agitar Technologies. 2014. AgitarOne | Agitar Technologies: Putting Java to the Test — agitar.com. <http://www.agitar.com/solutions/products/agitarone.html>. [Accessed 10-03-2025].
- [64] Valerio Terragni, Gunel Jahangirova, Paolo Tonella, and Mauro Pezzè. 2020. Evolutionary improvement of assertion oracles. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Virtual Event, USA) (ESEC/FSE 2020)*. Association for Computing Machinery, New York, NY, USA, 1178–1189. <https://doi.org/10.1145/3368089.3409758>
- [65] Valerio Terragni, Gunel Jahangirova, Paolo Tonella, and Mauro Pezzè. 2021. GAssert: A Fully Automated Tool to Improve Assertion Oracles. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 85–88. <https://doi.org/10.1109/ICSE-Companion52605.2021.00042>
- [66] Deepika Tiwari, Yogya Gamage, Martin Monperrus, and Benoit Baudry. 2024. PROZE: Generating Parameterized Unit Tests Informed by Runtime Data. In *2024 IEEE International Conference on Source Code Analysis and Manipulation (SCAM)*. 166–176. <https://doi.org/10.1109/SCAM63643.2024.00025>
- [67] Deepika Tiwari, Martin Monperrus, and Benoit Baudry. 2023. RICK: Generating Mocks from Production Data. In *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*. 464–466. <https://doi.org/10.1109/ICST57152.2023.00051>
- [68] Deepika Tiwari, Martin Monperrus, and Benoit Baudry. 2024. Mimicking Production Behavior with Generated Mocks. *IEEE Transactions on Software Engineering* (2024), 1–26. <https://doi.org/10.1109/TSE.2024.3458448>
- [69] Deepika Tiwari, Long Zhang, Martin Monperrus, and Benoit Baudry. 2022. Production Monitoring to Improve Test Suites. *IEEE Transactions on Reliability* 71, 3 (2022), 1381–1397. <https://doi.org/10.1109/TR.2021.3101318>
- [70] Edward R Tufte. 2001. *The visual display of quantitative information*. Vol. 2. Graphics press Cheshire, CT.
- [71] Julian Wachter, Deepika Tiwari, Martin Monperrus, and Benoit Baudry. 2025. Serializing Java Objects in Plain Code. arXiv:2405.11294 [cs.SE] <https://arxiv.org/abs/2405.11294>
- [72] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. 2020. On learning meaningful assert statements for unit test cases. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (Seoul, South Korea) (ICSE '20)*. Association for Computing Machinery, New York, NY, USA, 1398–1409. <https://doi.org/10.1145/3377811.3380429>
- [73] Tao Xie. 2006. Augmenting automatically generated unit-test suites with regression oracle checking. In *Proceedings of the 20th European Conference on Object-Oriented Programming (Nantes, France) (ECOOP'06)*. Springer-Verlag, Berlin, Heidelberg, 380–403. https://doi.org/10.1007/11785477_23
- [74] Tao Xie and David Notkin. 2004. Checking Inside the Black Box: Regression Testing Based on Value Spectra Differences. In *Proceedings. 20th IEEE International Conference on Software Maintenance*. IEEE Computer Society, Los Alamitos, CA, USA, 28–37. <https://doi.org/10.1109/ICSM.2004.1357787>
- [75] Lucas Zamprogno, Braxton Hall, Reid Holmes, and Joanne M. Atlee. 2023. Dynamic Human-in-the-Loop Assertion Generation. *IEEE Transactions on Software Engineering* 49, 4 (2023), 2337–2351. <https://doi.org/10.1109/TSE.2022.3217544>

Received 2025-09-12; accepted 2025-12-22